FarmaPredict · Borrador de la API para integrar (feature/api)

Nov 6, 2026 · @Alejandro Ronda Navazo

Punto de partida

La lógica del backend ya está en `main` y probada: `src/backend/datos.py` (carga de datos) y `src/backend/servicios.py` (búsqueda, ficha y sustitutos). Falta la capa HTTP: tres ficheros nuevos y dos funciones para el riesgo. El código de este documento está escrito pero **sin probar**: integrarlo y probarlo es la tarea.

Estructura en capas: `rutas.py` (HTTP) → `servicios.py` (lógica) → `datos.py` (ficheros). `rutas.py` no contiene lógica, solo traduce `None` a 404 y excepciones a códigos HTTP.

El contrato que debe cumplir la API está en `docs/ejemplos_endpoints.txt`.

Empezar la rama:

```
git switch main
git pull
git switch -c feature/api
python -m pip install -r requirements.txt   # fastapi y uvicorn ya están
```

Requisito previo: tener el worktree de `snapshots` y el `.env` configurados según el README (apartado "Cómo ejecutarlo").

Cambios en datos.py y servicios.py

El riesgo sale de `data/cleaned/riesgo_linea_base.csv`, que genera Francisco: una fila por CN con `cn;riesgo;nivel;horizonte_dias;modelo;fecha_calculo`. Mientras no exista, la API responde 503.

`src/backend/datos.py`: añadir `import csv` arriba, cambiar el import de config y añadir la función.

```
from src.config import DATA_CLEANED_DIR, DATA_RAW_DIR

RUTA_RIESGO = os.path.join(DATA_CLEANED_DIR, "riesgo_linea_base.csv")


@lru_cache(maxsize=1)
def cargar_riesgo() -> dict[str, dict] | None:
    """CN -> predicción de riesgo, o None si el modelo aún no ha generado el
```

```
fichero."""
    if not os.path.exists(RUTA_RIESGO):
        return None
    with open(RUTA_RIESGO, encoding="utf-8", newline="") as f:
        return {
            fila["cn"]: {
                "cn": fila["cn"],
                "riesgo": float(fila["riesgo"]),
                "nivel": fila["nivel"],
                "horizonte_dias": int(fila["horizonte_dias"]),
                "modelo": fila["modelo"],
                "fecha_calculo": fila["fecha_calculo"],
            }
            for fila in csv.DictReader(f, delimiter=";")
        }
```

Y añadir `cargar_riesgo` a la tupla de `recargar()`. Se usa `csv` y no pandas para que el CN se lea siempre como texto y no lleguen tipos de numpy a la respuesta JSON.

`src/backend/servicios.py`: añadir al final.

```
class ModeloNoDisponible(Exception):
    """El fichero de riesgo todavía no existe."""


def riesgo(cn: str) -> dict | None:
    tabla = datos.cargar_riesgo()
    if tabla is None:
        raise ModeloNoDisponible
    return tabla.get(cn)
```

Ficheros nuevos

`src/backend/esquemas.py`: modelos de respuesta. FastAPI valida con ellos que cada respuesta cumple el contrato y los muestra en `/docs`.

```
"""Modelos de respuesta de la API (el contrato de
docs/ejemplos_endpoints.txt)."""
from datetime import date

from pydantic import BaseModel


class Salud(BaseModel):
    status: str
```

```
    datos: dict[str, date]
    modelo_disponible: bool


class ResultadoBusqueda(BaseModel):
    cn: str
    nombre: str | None
    laboratorio: str | None
    comercializado: bool
    problema_activo: bool


class DCP(BaseModel):
    id: str | None
    nombre: str | None


class ProblemaSuministro(BaseModel):
    tipo: int | None
    descripcion: str
    fecha_inicio: date | None
    fecha_fin_prevista: date | None
    observaciones: str | None


class Ficha(BaseModel):
    cn: str
    nregistro: str | None
    nombre: str | None
    laboratorio: str | None
    dcp: DCP
    principio_activo: str | None
    comercializado: bool
    problema_suministro: ProblemaSuministro | None


class Opcion(BaseModel):
    cn: str
    nombre: str | None
    laboratorio: str | None
    dcp: str | None


class Sustitutos(BaseModel):
    cn: str
    dcp: DCP
```

```
    vias: list[str]
    sustitutos: list[Opcion]
    criterio_sustitutos: str
    alternativas: list[Opcion]
    criterio_alternativas: str


class Riesgo(BaseModel):
    cn: str
    riesgo: float
    nivel: str
    horizonte_dias: int
    modelo: str
    fecha_calculo: date
```

`src/backend/rutas.py`: los endpoints. Solo traducen entre HTTP y `servicios`.

```
"""Endpoints HTTP. Solo traducen entre HTTP y servicios: sin lógica de
negocio."""
from fastapi import APIRouter, HTTPException, Query

from src.backend import datos, servicios
from src.backend.esquemas import Ficha, ResultadoBusqueda, Riesgo, Salud,
Sustitutos

router = APIRouter()


@router.get("/health", response_model=Salud)
def health():
    return {
        "status": "ok",
        "datos": datos.fechas_datos(),
        "modelo_disponible": datos.cargar_riesgo() is not None,
    }


@router.get("/medicamentos", response_model=list[ResultadoBusqueda])
def buscar(q: str = Query(..., min_length=3), limite: int = Query(20, ge=1,
le=100)):
    return servicios.buscar(q, limite)


@router.get("/medicamentos/{cn}", response_model=Ficha)
def ficha(cn: str):
    resultado = servicios.ficha(cn)
```

```
    if resultado is None:
        raise HTTPException(status_code=404, detail="CN no encontrado")
    return resultado


@router.get("/medicamentos/{cn}/sustitutos", response_model=Sustitutos)
def sustitutos(cn: str):
    resultado = servicios.sustitutos(cn)
    if resultado is None:
        raise HTTPException(status_code=404, detail="CN no encontrado")
    return resultado


@router.get("/medicamentos/{cn}/riesgo", response_model=Riesgo)
def riesgo(cn: str):
    if cn not in datos.cargar_presentaciones():
        raise HTTPException(status_code=404, detail="CN no encontrado")
    try:
        resultado = servicios.riesgo(cn)
    except servicios.ModeloNoDisponible:
        raise HTTPException(status_code=503, detail="Modelo de riesgo no
disponible todavía")
    if resultado is None:
        raise HTTPException(status_code=404, detail="Sin predicción de riesgo
para este CN")
    return resultado
```

`src/backend/main.py`: punto de entrada. Carga los datos al arrancar, para que la primera petición no tarde y, si falta un fichero, el error salga al levantar el servidor y no en mitad de una demo.

```
"""Punto de entrada de la API.  ->  python -m uvicorn src.backend.main:app --
reload"""
from contextlib import asynccontextmanager

from fastapi import FastAPI

from src.backend import datos
from src.backend.rutas import router


@asynccontextmanager
async def lifespan(app: FastAPI):
    datos.cargar_presentaciones()
    datos.cargar_problemas_activos()
    datos.indice_por_dcp()
```

```
    datos.cargar_vias()
    yield


app = FastAPI(
    title="FarmaPredict API",
    version="0.3.0",
    description="Riesgo de desabastecimiento y sustitutos de medicamentos
(datos de AEMPS CIMA).",
    lifespan=lifespan,
)
app.include_router(router)
```

Probarlo

```
python -m uvicorn src.backend.main:app --reload
```

Abrir http://127.0.0.1:8000/docs (sirve también como captura de "backend activo" para el PDF), o desde otra terminal:

```
Invoke-RestMethod http://127.0.0.1:8000/health
Invoke-RestMethod "http://127.0.0.1:8000/medicamentos?q=becloforte"
Invoke-RestMethod http://127.0.0.1:8000/medicamentos/917070 | ConvertTo-Json
-Depth 4
Invoke-RestMethod http://127.0.0.1:8000/medicamentos/917070/sustitutos |
ConvertTo-Json -Depth 4
Invoke-RestMethod http://127.0.0.1:8000/medicamentos/917070/riesgo
Invoke-RestMethod http://127.0.0.1:8000/medicamentos/000000
```

Petición	Resultado esperado
`/health`	`status: ok`, fechas de los datos y `modelo_disponible: false` (hasta que exista el CSV de riesgo)
`?q=becloforte`	Becloforte 250 en la lista
`/917070`	Ficha con problema de suministro de tipo 5 desde el 04/11
`/917070/sustitutos`	`sustitutos` vacío y 3 `alternativas` (beclometasonas inhaladas de 50 y 100 µg)
`/917070/riesgo`	**503** mientras no haya modelo (es lo correcto)

Petición	Resultado esperado
`/000000`	**404** "CN no encontrado"
`?q=ab`	**422** (mínimo 3 caracteres)

Hecho cuando:

- [ ] Todas las peticiones de la tabla devuelven lo esperado
- [ ] `/docs` muestra los 5 endpoints con sus esquemas
- [ ] Cuando Francisco genere `riesgo_linea_base.csv`, `/riesgo` devuelve 200 y `/health` marca `modelo_disponible: true`
- [ ] Tests con `TestClient` de FastAPI para los códigos 200, 404, 422 y 503
- [ ] PR de `feature/api` a `main` revisada por otra persona